

* BT:

```
1) interface Mail {  
    void sendMail();  
    void log();  
}
```

```
class MailImpl implements Mail {
```

@Override

```
    public void sendMail() {  
        System.out.println("Đang gửi mail...");  
    }
```

}

@Override

```

    public void log() {
        System.out.println("Pang ghi log...");
    }

    class Main {
        public static void main (String [] args) {
            Mail Impl mail = new Mail Impl();
            mail.send Mail();
            mail.log();
        }
    }

```

297 ^{Shape} ~~interface~~ interface Shape {

```

    double get Area();
    double get Perimeter();
    void draw();
    public
    class Rectangle implements Shape {
        double width;
        double height;
    }
    Rectangle (double width, double height) {
        this.width = width;
        this.height = height;
    }
    @Override
    public double getArea() {

```

```
} @Override
```

```
public double getPerimeter () {
```

```
return 2 * (width + height);
```

```
} @Override
```

```
public void draw () {
```

```
System.out.println("Drawing Rectangle");
```

```
}
```

Circle - class Circle implements Shape {

```
private double radius;
```

```
public Circle (double radius) {
```

```
this.radius = radius;
```

```
} @Override
```

```
public double getArea () {
```

```
return Math.PI * radius * radius;
```

```
} @Override
```

```
public double getArea () {
```

```
return Math.PI * radius;
```

```
} @Override
```

```
public void draw () {
```

```
System.out.println("Drawing Circle");
```

* Main

```
public class Main Shape of  
{  
    public static void main(String[] args) {  
        Shape r = new Rectangle(5, 4);  
        Shape c = new Circle(3);  
        r.draw();  
        System.out.println("Area: " + r.getArea());  
        System.out.println("Perimeter: " + r.getPerimeter());  
        c.draw();  
        System.out.println("Area: " + c.getArea());  
        System.out.println("Perimeter: " + c.getPerimeter());  
    }  
}
```

BT3

class HangSanXuat {

String tenHangSanXuat;

String tenQuocGia;

HangSanXuat(String tenHangSanXuat, String tenQuocGia) {

this.tenHangSanXuat = tenHangSanXuat;

this.tenQuocGia = tenQuocGia;

}

public String toString() {

return tenHangSanXuat + " - " + tenQuocGia

}

abstract class PhươngTiền DiChuyen {

String loạiPhien;

HangSanXuat hangSanXuat;

PhươngTiền DiChuyen (String loạiPhien, HangSanXuat hangSanXuat) {

this.loạiPhien = loạiPhien;

this.hangSanXuat = hangSanXuat;

}

String layTenHangSanXuat() {

return hangSanXuat.tenHangSanXuat;

}

void DkBatDau() {

System.out.println("Đã bắt đầu di chuyển");

} void TangToc() {

System.out.println("Tăng tốc");

} void dung lai() {

System.out.println("Dừng lại");

}

} abstract double layVanToc();

```

class Maybay extends PhươngTruyền {
    String loai: Nhiên Liệu;
    Maybay (Hàng San Xuất hàng San Xuất, String loai: Nhiên Liệu) {
        super ("Maybay", hàng San Xuất);
        this.loai: Nhiên Liệu = loai: Nhiên Liệu;
    }
    void cat canh () {
        System.out.println ("Mb cat canh");
    }
    void ha canh () {
        System.out.println ("Mb ha canh");
    }
}

```

@Override

```

double lay ban tốc () {
    return 800;
}

```

```

class Xe Oto extends PhươngTruyền {

```

```

    String, loai: Nhiên Liệu;

```

```

    Xe Oto (Hàng SX hàng SX, String, loai: Nhiên Liệu) {

```

```

        super ("Xe Oto", hàng SX);

```

```

        this.loai: Nhiên Liệu = loai: Nhiên Liệu;
    }

```

@Override

```
double layVanToc() {  
    return 120;  
}
```

```
}
```

```
class XeDap extends PhuongTienDiChuyen {
```

```
    XeDap(HangSanXuat hangSX) {
```

```
        super("XeDap", hangSX);
```

```
    }
```

```
    @Override
```

```
        double layVanToc() {
```

```
            return 25;
```

```
        }
```

```
public class Main
```

```
{  
    public static void main(String[] args)
```

```
{  
        HangSX h1 = new HangSX("Da Boring", "My");
```

```
        HangSX h2 = new HangSX("Toyota", "Nhật");
```

```
        HangSX h3 = new HangSX("Thong Nhat", "Viet Nam");
```

```
        MayBay mb = new MayBay(h1, "Xang");
```

```
        XeOto oto = new XeOto(h2, "Xang");
```

```
        XeDap xd = new XeDap(h3);
```

```
        System.out.println("Van doc may bay: " + mb.layVanToc());  
    }
```

System.out.println("Can xac doi" + oto.
Lay Van TOE C1);

System.out.println("Can xe dap" + xol.layVan TOE C1);

} }

BT 4: class Car {

protected String color;

protected double speed;

public Car (String color) {

this.color = color;

this.speed = 0;

}

void accelerate () {

speed += 10;

}

public String getColor () {

return color;

}

public double getSpeed () {

return speed;

}

}

```
class BMW extends Car {  
    public BMW (String color) {  
        super (color);
```

```
    } @Override
```

```
    void accelerate () {
```

```
        speed += 20;
```

```
    } @Override
```

```
    public String toString () {
```

```
        return "BMW - Color: " + color + " Speed: " + speed;
```

```
    }
```

```
class Civic extends Car {
```

```
    public Civic (String color) {
```

```
        super (color);
```

```
    } @Override
```

```
    void accelerate () {
```

```
        speed += 15;
```

```
    } @Override
```

```
    public String toString () {
```

```
        return "Civic - Color: " + color + " Speed: " + speed;
```

```
    }
```

```
public class Main {  
    public static void main (String[] args)
```

```
    {  
        BMN b = new BMN ("black");
```

```
        Circle c = new Circle ("Red");
```

```
        b.accelerate ();
```

```
        c.accelerate ();
```

```
        System.out.println (b.toString ());
```

```
        System.out.println (c.toString ());
```

```
    }  
}
```

```
BTS: interface Moveable {
```

```
    void move up ();
```

```
    void move down ();
```

```
    void move left ();
```

```
    void move right ();
```

```
class Moveable Point implements Moveable {
```

```
    int x;
```

```
    int y;
```

```
    int x speed;
```

```
    int y speed;
```

```
    Moveable Pointa (int x, int y, int x Speed, int y Speed) {
```

```

    public void move up () {
        y += ySpeed;
    }
    public void move down () {
        y -= ySpeed;
    }
    public void move left () {
        x -= xSpeed;
    }
    public void move right () {
        x += xSpeed;
    }
}

public String toString () {
    return "Point (" + x + ", " + y + ")";
}
}

```

```

class Moveable Circle implements Moveable {
    int radius;
    Moveable Paint center;
    public void move up () {
        center. move up ();
    }
    public void move down () {
        center. movedown ();
    }
    public void move Left () {
        center - move left ();
    }
}

```

```

    }
    public void move right () {
        center . move right ();
    }
    public String toString () {
        return "Circle center : " + center + " radius " + radius;
    }
}

```

```

public class Main {
    public static void main (String [] args)
    {

```

```

        MoveablePoint p = new MoveablePoint (2, 3, 1, 1);
        System.out.println (p);

```

```

        p . move up ();

```

```

        p . move right ();

```

```

        System.out.println (p);

```

```

        MoveableCircle c = new MoveableCircle (p, 5);

```

```

        System.out.println (c);

```

```

        c . move down ();

```

```

        c . move left ();

```

```

        System.out.println (c);
    }
}

```